EXPLAIN ARRAY addressing calculation 1d 2d 3d band so on general formula ; base_address+...

Alright, let's break **array addressing calculation** into a very simple, clear idea 👍

🔷 1D Array Address Formula

For a **1D array**, memory is continuous.

📌 Formula:

$$\text{Address}(A[i]) = \text{Base Address} + (i - L) \times \text{size}$$

🧠 Meaning:

- **Base Address** = address of first element
- **i** = index you want
- **L** = lower bound (usually 0 in most languages)
- **size** = size of one element (e.g., 4 bytes for int)

✅ Example:

Array: `A[0…4]`, Base = 1000, size = 4 bytes

Find address of `A[3]`:

$$1000 + (3 - 0) \times 4 = 1000 + 12 = 1012$$

🔷 2D Array Address Formula

Now things depend on **storage order**:

▶️ Row-Major Order (most common, like C/C++)

Rows stored one after another.

📌 Formula:

$$\text{Address}(A[i][j]) = \text{Base} + [(i - L_r) \times N_c + (j - L_c)] \times \text{size}$$

🧠 Meaning:

- N_c = number of columns
- L_r, L_c = lower bounds

✅ Example:

Array: `A[2][3]`, Base = 1000, size = 4
Find `A[1][2]`:

$$1000 + [(1 \times 3) + 2] \times 4 = 1000 + (5 \times 4) = 1020$$

▶️ Column-Major Order (used in Fortran, MATLAB)

Columns stored first.

📌 Formula:

$$\text{Address}(A[i][j]) = \text{Base} + [(j - L_c) \times N_r + (i - L_r)] \times \text{size}$$

🔷 3D Array Address Formula

Now we extend the same idea.

▶️ Row-Major Order

📌 Formula:

$$\text{Address}(A[i][j][k]) = \text{Base} + [(i - L_1) \times N_2 \times N_3 + (j - L_2) \times N_3 + (k - L_3)] \times \text{size}$$

🧠 Meaning:

- N_2 = size of 2nd dimension
- N_3 = size of 3rd dimension

🔷 General Formula (n-D Array) ⭐

📌 Row-Major:

$$\text{Address} = \text{Base} + \text{size} \times \left[\sum (index - lower\ bound) \times product\ of\ sizes\ of\ next\ dimensions\right]$$

🧠 Simple Intuition:

👉 Think like this:

- **1D** → just move forward
- **2D** → skip full rows + move in row
- **3D** → skip full blocks + rows + columns

🔷 Super Simple Memory Trick 🧠

- **1D:** move → `i`
- **2D:** move → `i × columns + j`
- **3D:** move → `i × (rows×cols) + j × cols + k`

Then multiply by size and add base.

🔥 Final Compact Formulas

✅ 1D:

$$B + (i - L) \times w$$

✅ 2D (Row-Major):

$$B + [(i \times N_c) + j] \times w$$

✅ 3D (Row-Major):

$$B + [(i \times N_2 \times N_3) + (j \times N_3) + k] \times w$$

If you want, I can give **step-by-step solved questions** (easy → exam level) or a **shortcut trick to solve in seconds**.

INDEXING IN 2D AND 3D FORMULA ALSO COLUMN MAJOR FORMULA IN 3D AND COLUMN MAJOR FORMULA GENERALLY AND ROW MAJOR FORMULA GENERALLY FOR ADDRESSING

Good — now let's organize **ALL formulas (2D, 3D, and GENERAL n-D)** in a clean, exam-ready way 👍

🔷 Notation (Important First)

- **B** = Base address
- **w** = size of each element
- **i, j, k** = indices
- **L_1, L_2, L_3** = lower bounds
- **N_1, N_2, N_3** = sizes of dimensions

🔷 2D ARRAY INDEXING

✅ Row-Major Order (C/C++)

👉 Rows stored first

📌 Formula:

$$\text{Address}(A[i][j]) = B + w \times [(i - L_1) \times N_2 + (j - L_2)]$$

✅ Column-Major Order (Fortran, MATLAB)

👉 Columns stored first

📌 Formula:

$$\text{Address}(A[i][j]) = B + w \times [(j - L_2) \times N_1 + (i - L_1)]$$

🔷 3D ARRAY INDEXING

✅ Row-Major Order

👉 First dimension changes slowest

📌 Formula:

$$\text{Address}(A[i][j][k]) = B + w \times [(i - L_1) \times N_2 \times N_3 + (j - L_2) \times N_3 + (k - L_3)]$$

✅ Column-Major Order

👉 First dimension changes fastest

📌 Formula:

$$\text{Address}(A[i][j][k]) = B + w \times [(k - L_3) \times N_1 \times N_2 + (j - L_2) \times N_1 + (i - L_1)]$$

🔷 GENERAL FORMULA (n-D ARRAY)

✅ Row-Major (GENERAL) ⭐

👉 Leftmost index slowest, rightmost fastest

$$\text{Address} = B + w \times \sum_{p=1}^{n} \left[(i_p - L_p) \times \prod_{q=p+1}^{n} N_q\right]$$

🧠 Simple Meaning:

- Multiply each index by **product of sizes of dimensions to its right**

✅ Column-Major (GENERAL) ⭐

👉 Leftmost index fastest

$$\text{Address} = B + w \times \sum_{p=1}^{n} \left[(i_p - L_p) \times \prod_{q=1}^{p-1} N_q\right]$$

🧠 Simple Meaning:

- Multiply each index by **product of sizes of dimensions to its left**

🔥 SUPER SIMPLE SHORTCUT (Exam Trick)

▶️ Row-Major

👉 Move like:

```
i → full blocks
j → rows
k → columns
```

▶️ Column-Major

👉 Reverse thinking:

```
i → fastest
j → next
k → slowest
```

🔷 Quick Comparison

Type	Formula Idea
Row-Major	Right side dimensions multiply
Column-Major	Left side dimensions multiply

🔥 Final Memory Trick 🧠

- **Row-Major:** → "RIGHT matters"
- **Column-Major:** → "LEFT matters"

If you want, I can:
✅ Solve tricky past paper questions
✅ Give MCQs
✅ Show visualization (very helpful for exams)

3D AND 2D AND BIGGER ARRAYS USING FOR LOOP INDEXING USING NEW KEYWORD TO DECLARE THE ARRAY AND STORE DATA IN IT

Got it — you want to understand **how 2D, 3D (and bigger) arrays are created using `new` and accessed using loops (indexing)**. I'll explain clearly with C++ examples 👇

🔷 1. 2D Array using `new` + Loop Indexing

✅ Declaration (Dynamic 2D Array)

```
int rows = 3, cols = 4;

// create array of pointers (rows)
int** arr = new int*[rows];

// allocate each row
for(int i = 0; i < rows; i++) {
    arr[i] = new int[cols];
}
```

✅ Storing Values (Indexing with loops)

```
for(int i = 0; i < rows; i++) {
    for(int j = 0; j < cols; j++) {
        arr[i][j] = i + j;   // example value
    }
}
```

✅ Accessing / Printing

```
for(int i = 0; i < rows; i++) {
    for(int j = 0; j < cols; j++) {
        cout << arr[i][j] << " ";
    }
    cout << endl;
}
```

🔷 2. 3D Array using `new` + Loop Indexing

✅ Declaration

```
int x = 2, y = 3, z = 4;

int*** arr = new int**[x];

for(int i = 0; i < x; i++) {
    arr[i] = new int*[y];

    for(int j = 0; j < y; j++) {
        arr[i][j] = new int[z];
    }
}
```

✅ Storing Values

```
for(int i = 0; i < x; i++) {
    for(int j = 0; j < y; j++) {
        for(int k = 0; k < z; k++) {
            arr[i][j][k] = i + j + k;
        }
    }
}
```

✅ Accessing / Printing

```
for(int i = 0; i < x; i++) {
    for(int j = 0; j < y; j++) {
        for(int k = 0; k < z; k++) {
            cout << arr[i][j][k] << " ";
        }
        cout << endl;
    }
    cout << "----" << endl;
}
```

🔷 3. General n-D Idea (Concept)

👉 You **cannot easily write generic n-D `new` code** without recursion/templates, but conceptually:

- Add one more pointer level for each dimension
- Add one more loop for indexing

🧠 Pattern

Dimension	Pointer Type	Loops
1D	`int*`	1 loop
2D	`int**`	2 loops
3D	`int***`	3 loops
n-D	`int****...`	n loops

🔷 4. Better Method (Continuous Memory) ⭐

Instead of pointer-of-pointer (slow), use **single block (like real memory)**

✅ 2D Continuous Allocation

```
int* arr = new int[rows * cols];

// access:
arr[i * cols + j] = 5;
```

✅ 3D Continuous Allocation

```
int* arr = new int[x * y * z];

// access:
arr[i * (y*z) + j * z + k] = 10;
```

🔷 5. Why Continuous Method is Better

✔️ Faster (cache friendly)
✔️ Matches real memory layout
✔️ Easy address calculation (same formulas you studied)

🔥 Final Understanding

👉 **Indexing logic = same as address formula**

- 2D:

```
index = i * cols + j
```

- 3D:

```
index = i * (y*z) + j * z + k
```

🔷 6. Important (Memory Deallocation)

2D:

```
for(int i = 0; i < rows; i++) {
    delete[] arr[i];
}
delete[] arr;
```

3D:

```
for(int i = 0; i < x; i++) {
    for(int j = 0; j < y; j++) {
        delete[] arr[i][j];
    }
    delete[] arr[i];
}
delete[] arr;
```

🔥 Quick Summary

- `new` + pointers → flexible but complex
- Continuous array → simple + matches formulas
- Indexing = **address calculation without base & size**

If you want, I can:
✅ Convert these into **exam-ready short answers**
✅ Show **common mistakes students make**
✅ Give **MCQs + solved numericals**